Proof of Concept Documentation

Introduction

This work presents a **real-time voice chatbot system** designed for **noisy environments and multilingual (Hindi/Hinglish) interaction**, based on a **3-layer architecture**.

The goal was to:

- Maintain **low-latency noise suppression (<200 ms)**
- Ensure **cost efficiency**
- Build **robust speech detection**
- Enable **safe interaction via validation logic**

Research-Based Conclusion of the 3-Layer Architecture:

1. Findings from Noise Reduction Research

From analysis of:

- RNNoise (GRU-based real-time denoising)
- DeepFilterNet, DNS Challenge papers
- Real-time DSP constraints (STFT, frame processing)

Concluded:

- Real-time noise suppression requires **frame-based processing (10–20 ms)**
- Cloud-based suppression introduces **network + buffering latency**
- Low-latency models like RNNoise are suitable for **edge execution**

Founded:

- Noise suppression must be handled at the **User Side (Edge Layer)** to meet <200 ms constraints.

2. Findings from Speech Recognition Systems

From studying:

- Azure Speech SDK
- Google Speech-to-Text
- Whisper, NVIDIA Riva

Founded:

- ASR models require **high compute and scalability**
- Cloud systems provide:
    - multilingual support
    - streaming inference
    - model updates

Conclusion:

Speech recognition and language understanding must reside in the **Cloud Layer.**

3. Findings from Indian Linguistic Conditions

From analysis of:

- AI4Bharat Indic ASR
- Hinglish/code-switching behavior

Founded:

- Mixed-language inputs reduce ASR confidence.
- Noise + code-switching increases ambiguity.

Conclusion:

A post-processing control layer is required to verify output before execution.

5. Findings from Cost and Latency Analysis

From evaluating:

- Cloud pricing models (Azure Speech, OpenAI)
- Streaming vs filtered audio

Observed:

- Sending raw audio increases:
    - bandwidth
    - compute cost
- Silence/noise contributes no value

Conclusion:

Preprocessing (VAD + denoising) must be done at **User Side to reduce cost.**

6. Findings from Real-Time Constraints

From latency breakdown:

- Frame processing: 10 ms
- RNNoise: ~1 ms/frame
- Network + ASR: dominant latency

Concluded:

- Only **lightweight operations can run on edge**.
- Heavy models must remain in cloud.

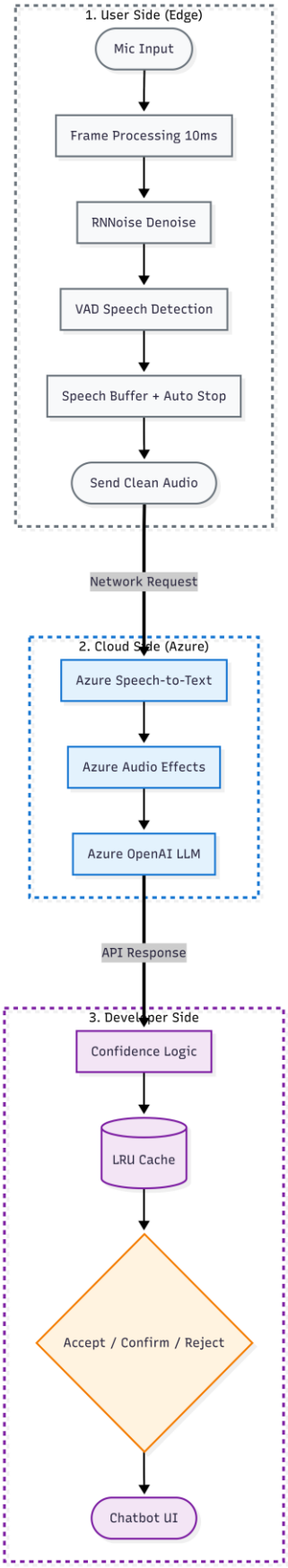
Derived Architecture from findings:

User Side → Cloud Side → Developer Side

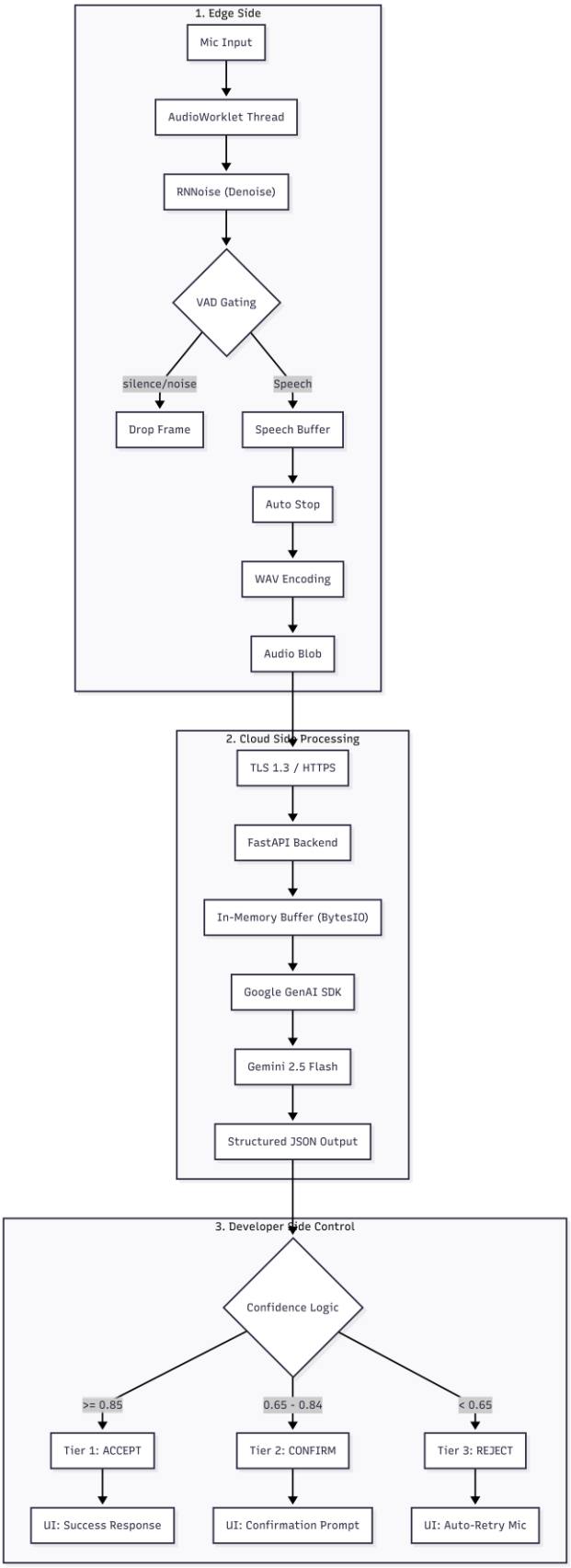
(Noise + VAD) (ASR + AI) (Control + Safety)

Through analysis of real-time noise suppression techniques, speech recognition systems, linguistic challenges, and cost-latency trade-offs, I have derived a 3-layer architecture where edge processing handles noise and filtering, cloud systems handle speech understanding, and a developer-controlled layer ensures safe and reliable execution.

Architecture Diagram:



From the above Idea I have developed a prototype as a demonstration for POC, I have integrated the Gemini API Key in the cloud side as this is just a prototype. The architecture I have followed for developing this POC is:



For the prototype, I integrated the Gemini API in the cloud layer; any cloud service can be used for the final build.

VaakBot Voice System — Documentation for the Codebase

Brief Overview:

This document explains the **new implementation** of VaakBot: a real-time voice chatbot pipeline that processes Hindi/Hinglish audio, suppresses background noise on the edge, sends only speech-worthy audio to the backend, and returns a structured AI response with confidence-based UI handling.

The system keeps the same **3-layer architecture** we designed originally:

1. **User Side (Edge)** — microphone capture, frame processing, RNNoise denoising, VAD gating, speech buffering, auto-stop.
2. **Cloud Side** — FastAPI backend, in-memory audio handling, Gemini-based audio understanding, structured JSON response.
3. **Developer Side** — confidence thresholds, ACCEPT / CONFIRM / REJECT decision logic, and chatbot UI handling.

Original Idea vs. Current Implementation:

Original 3-layer architecture:

- **User Side:** capture audio and prepare it for processing.
- **Cloud Side (original design):** Azure Speech / Azure Audio Effects / Azure OpenAI.
- **Developer Side:** validation, confidence checks, and safe execution.

Current implementation:

The cloud layer in the current prototype was implemented using **Google GenAI + Gemini 2.5 Flash** as the working backend, while preserving the same architecture and logic flow.

The important point is that the **architecture did not change**; only the cloud service provider used for the prototype changed.

The new frontend is a full conversational UI with a real-time audio pipeline.

User Interface:

The UI contains:

- a chatbot header,
- a mic button to start/stop recording,
- a waveform canvas,
- a live VAD indicator,
- a silence auto-stop bar,
- a response card showing transcript, confidence, and verdict,
- latency metrics and logs.

Audio Capture Layer:

The code captures audio using the browser microphone and an **AudioWorklet** thread. AudioWorklet is the correct choice for real-time audio because it runs in a low-latency audio processing thread rather than on the main UI thread.

RNNoise Denoising:

The frontend uses **RNNoise WASM** for noise suppression. RNNoise is a lightweight GRU-based denoiser designed for real-time speech enhancement.

The code loads RNNoise from WebAssembly, initializes the state, and processes audio frame-by-frame.

Frame Size and Timing

The frontend is configured for:

- **Sample rate:** 48,000 Hz
- **Frame size:** 480 samples
- **Frame duration:**

$$\text{Frame Duration} = \frac{480}{48000} = 0.01 \text{ s} = 10 \text{ ms}$$

So the system processes audio in **10 ms frames**.

VAD Gating

The system uses a VAD decision based on RNNoise output probability.

- If the frame is speech-like, it is kept.
- If it is silence/noise-like, it is dropped.

This reduces unnecessary cloud traffic and lowers cost.

Speech-Based Auto Stop

The recording does **not** stop on pure silence alone. It stops when there is **no speech for a fixed duration**, even if background noise is still present.

In the current implementation:

- SILENCE_FRAMES_MAX = 200
- each frame = 10 ms

So:

$$200 \times 10 \text{ ms} = 2000 \text{ ms} = 2 \text{ seconds}$$

The recording stops if there **is no speech for 2 seconds.**

WAV Encoding and Upload

Before sending audio to the backend, the frontend:

- merges buffered frames,
- converts Float32 samples to PCM 16-bit,
- creates a WAV blob,
- uploads it to the backend using fetch().

What this frontend proves

This file proves that the core edge logic works:

- real-time frame processing,
- denoising,
- speech filtering,
- speech-based stopping,
- browser UI updates,
- backend-ready WAV packaging.

The backend is implemented in **FastAPI** and uses the Google GenAI SDK.

Backend responsibilities

The backend:

- receives uploaded audio,
- validates or re-encodes it as WAV,
- places audio in memory,
- sends it to Gemini,
- receives structured output,
- calculates confidence-based verdicts,
- returns a JSON response to the UI.

In-Memory Processing

The backend uses:

- io.BytesIO()
- wave.open(..., 'wb')

This means the audio is processed in **RAM**, not written to disk.

This improves speed and keeps the request temporarily adding a layer for security advantage as the data is never written to the persistent storage.

Structured Output

The backend defines a response schema:

- transcript
- confidence_score
- ui_confirm
- ui_reject

This is important because the model response is not free-form text; it is constrained into a predictable JSON-like structure.

Confidence Logic:

The backend maps confidence into a verdict:

- **ACCEPT** if the score is high,
- **CONFIRM** for medium confidence,
- **REJECT** for low confidence.

This supports safe conversational control.

Audio Normalization:

The backend enforces a consistent WAV format in memory. This avoids format mismatches and keeps the pipeline stable.

What this backend proves:

This file proves that the cloud side is able to:

- accept audio,
- process it in memory,
- obtain structured AI output,
- apply a decision policy,
- send back UI-ready results.

Audio and Latency Calculations:

Frame duration:

Using 48 kHz audio and 480 samples per frame:

$$\text{Frame Duration} = \frac{480}{48000} = 10 \text{ ms}$$

Speech validation window:

The code waits for 30 consecutive speech frames before it considers speech to be stable:

$$30 \times 10 \text{ ms} = 300 \text{ ms}$$

This helps avoid false positives from short noise bursts.

No-speech stop window:

The recording stops after 200 no-speech frames:

$$200 \times 10 \text{ ms} = 2000 \text{ ms} = 2 \text{ s}$$

RNNoise latency:

The frontend measures RNNoise processing time per frame and accumulates it. The expected denoising time is approximately **~1 ms per frame** in the browser-sized POC.

Data reduction from VAD:

Because only speech frames are buffered and sent, the cloud receives much less useless audio. If speech occupies about 40% of the stream, then about **60% of the traffic is reduced**.

Future Integrations:

Silero VAD: Silero VAD is a strong future upgrade for speech detection.

Safe sources:

- https://github.com/snakers4/silero-vad
- https://pytorch.org/hub/snakers4_silero-vad_vad/

It can be integrated later if a separate neural VAD model is required instead of using RNNoise-derived gating.

LRU Cache: An LRU cache can be added in the developer layer for repeated high-confidence queries.

Concept:

- if the confidence is extremely high,
- and the query matches with the cached intent,
- then the response can be returned without going back to the cloud.

This would reduce:

- latency,
- cloud usage,
- token cost.

This cache is a **future optimization idea** and is not part of the current code.

What Is Implemented Now vs. What Is Future Work:

Implemented now:

- real-time audio capture,
- AudioWorklet processing,
- RNNoise WASM denoising,
- speech/noise gating,
- no-speech auto stop,
- WAV generation,
- FastAPI backend,
- in-memory audio handling,
- Gemini structured output,
- confidence-based response logic,
- chatbot UI response handling.

Future work:

- Silero VAD integration,
- LRU cache optimization,
- Azure cloud deployment,
- deeper metrics logging,
- production-scale observability.

Conclusion:

This codebase correctly implements the core 3-layer idea in a working form:

- **User Side:** real-time edge noise suppression and speech gating,
- **Cloud Side:** in-memory AI processing and structured transcript output,
- **Developer Side:** confidence-based safe decision handling.

The current implementation is a strong technical proof of the architecture, and the planned future integrations (Silero VAD and LRU cache) fit naturally into the same design.

Short Summary:

This codebase demonstrates a practical real-time voice chatbot pipeline with low-latency edge audio processing, structured cloud inference, and safe developer-layer control.

(End of Documentation)